



Python Data Processing & Asynchronous ETL Pipeline

이 프로젝트는 대용량 로그 처리부터 Pydantic 검증, asyncio 비동기 수집기, 그리고 비동기 ETL 파이프라인 구축 및 단위 테스트 실습 내용을 담고 있습니다.



목차

1. 실습 1. 대용량 로그 스트리밍 집계
2. 실습 2. Pydantic v2 중첩 스키마 검증
3. 실습 3. asyncio 기반 비동기 수집기
4. 종합 실습 1. 비동기 ETL 파이프라인
5. 추가 과제

실습 1. 대용량 로그 스트리밍 집계

STEP 0. 데이터 형태 확인

```
=====
STEP 0. 데이터 형태 확인
=====
ip,timestamp,method,path,status,bytes,user_agent
72.138.86.80,2025-03-01T00:00:05,POST,/api/v1/search,404,122887,Mozilla/5.0 (Windows NT 10.0)
113.211.16.180,2025-03-01T00:00:05,GET,/products/detail,200,178381,python-httpx/0.27.2
6.38.134.252,2025-03-01T00:00:05,GET,/products/detail,200,133763,python-httpx/0.27.2
194.249.158.140,2025-03-01T00:00:10,GET,/,404,165382,Mozilla/5.0 (Macintosh)
```

STEP 1. 파일 -> 딕셔너리 제너레이터

```
=====
STEP 1. 파일 -> 레코드 제너레이터
=====
{'ip': '72.138.86.80', 'timestamp': '2025-03-01T00:00:05', 'method': 'POST', 'path': '/api/v1/search', 'status': '404', 'bytes': '122887', 'user_agent': 'Mozilla/5.0 (Windows NT 10.0)'}
{'ip': '113.211.16.180', 'timestamp': '2025-03-01T00:00:05', 'method': 'GET', 'path': '/products/detail', 'status': '200', 'bytes': '178381', 'user_agent': 'python-httpx/0.27.2'}
{'ip': '6.38.134.252', 'timestamp': '2025-03-01T00:00:05', 'method': 'GET', 'path': '/products/detail', 'status': '200', 'bytes': '133763', 'user_agent': 'python-httpx/0.27.2'}
```

STEP 2 + 3. 집계 코드 작성

```
=====
STEP 2 + 3. 파일 읽기(yield) + Counter
=====
by_status 5개 : [('200', 159884), ('500', 8126), ('301', 8055), ('404', 8007), ('302', 8007)]
by_path 5개 : [('/static/app.js', 20204), ('/health', 20152), ('/checkout', 20128), ('/cart', 20036), ('/products/detail', 20030)]
by_hour 5개 : [('03', 8769), ('09', 8729), ('00', 8685), ('07', 8674), ('15', 8672)]
by_ip 5개 : [('83.3.180.55', 2), ('173.10.79.216', 2), ('84.136.62.63', 2), ('205.212.216.202', 2), ('186.215.89.61', 2)]
```

STEP 4. 5xx 비율 계산

```
=====
STEP 4. 5xx 비율 계산
=====
5xx: 16047건 (8.0%)
```

STEP 5. fold 패턴

```
=====
STEP 5. fold 패턴 - functools.reduce 활용
=====
200000
Counter({'200': 159884, '500': 8126, '301': 8055, '404': 8007, '302': 8007, '503': 7921})
```

STEP 6. 리포트 및 상위 IP 출력

```
=====
리포트 및 상위 IP
=====

총 요청 수 : 200,000
5xx 오류율 : 8.0%

-- 인기 경로 TOP 5 --
/static/app.js      20,204
/health             20,152
/checkout           20,128
/cart               20,036
/products/detail    20,030

-- 인기 접속 시간 TOP 5 --
03                  8,769
09                  8,729
00                  8,685
07                  8,674
15                  8,672

-- 접속 상위 IP TOP 5 --
83.3.180.55         2
173.10.79.216       2
84.136.62.63        2
205.212.216.202     2
186.215.89.61       2
```

실습 2. Pydantic v2 중첩 스키마 검증

STEP 0 · 오염 데이터 확인

```
=====
STEP 0. 데이터 오염 확인
=====
전체 건수: 40
{
  "id": 1,
  "username": "user_001",
  "email": "user1@example.com",
  "age": 30,
  "is_active": false,
  "signup_date": "2024-07-02",
  "profile": {
    "country": "KR",
    "tier": "pro",
    "score": 92.91
  },
  "tags": []
}
0 {'id': 'int', 'username': 'str', 'email': 'str', 'age': 'int', 'is_active': 'bool', 'signup_date': 'str', 'profile': 'dict', 'tags': 'list'}
1 {'id': 'int', 'username': 'str', 'email': 'str', 'age': 'int', 'is_active': 'bool', 'signup_date': 'str', 'profile': 'dict', 'tags': 'list'}
2 {'id': 'int', 'username': 'str', 'email': 'str', 'age': 'int', 'is_active': 'bool', 'signup_date': 'str', 'profile': 'dict', 'tags': 'list'}
```

- **7번 유저** : 나이가 음수값(-5)로 설정됨
- **13번 유저** : 이메일 값이 "not-an-email"으로 형식을 벗어남
- **21번 유저** : 이메일 태그 없음
- **29번 유저** : score가 150.0으로 100점 초과

STEP 1 ~ 4. 모델 생성 및 규약 설정

```

class UserProfile(BaseModel):
    country: str = Field(..., min_length=1)
    tier: str = Field(..., min_length=1)
    score: float = Field(..., ge=0, le=100) # 29번 유저 차단: 최소 0점, 최대 100점

# STEP 1, 2, 3, 4
class User(BaseModel):
    id: int
    username: str = Field(..., min_length=1)

    # 💡 21번 유저 차단: 기본값(default)을 주지 않아 필드가 없으면 ValidationError를 던짐
    email: str

    # 💡 7번 유저 차단: 나이는 무조건 0보다 커야 함 (gt=0)
    age: int = Field(..., gt=0)

    is_active: bool
    signup_date: str
    profile: UserProfile
    tags: List[str] = []

    # 💡 13번 유저 차단: 이메일 필드에 최소한 @와 .이 들어간 정상 형식인지 정규식 검증
    @field_validator("email")
    @classmethod
    def validate_email_format(cls, v: str) -> str:
        email_regex = r"^[a-zA-Z0-9_+]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-]+$"
        if not re.match(email_regex, v):
            raise ValueError("올바르지 않은 이메일 형식입니다.")
        return v

```

STEP 5 · 40건을 돌리며 유효/무효로 나누기

```

=====
STEP 5. 40건 유효/무효 검증
=====
전체 40건 → 유효 36 / 오류 4

```

STEP 6 · 탈락 사유를 표로 출력 (확장 과제)

```

=====
STEP 6. 탈락 사유를 표로 출력
=====
행   필드      사유
6   age       Input should be greater than 0
12  email      Value error, 올바르지 않은 이메일 형식입니다.
20  email      Field required
28  profile.scoreInput should be less than or equal to 100

```

실습 3. asyncio 기반 비동기 수집기

STEP 0. 동기 버전

```

=====
STEP 0. fetch_sync : 동기
=====
동기 처리 소요 시간: 6.03초

```

STEP 1. async/await 비동기

```

=====
STEP 1. fetch : 비동기 단건
=====
{'id': 1, 'ok': True}
비동기 단건 소요 시간: 0.10초

```

STEP 2. gather로 60개 동시에 던지기

```
=====
STEP 2. fetch + gather : 무제한 비동기
=====
60
무제한 비동기 소요 시간: 0.11초
```

STEP 3. 백프레서 — Semaphore 로 동시 요청 수 제한

```
=====
STEP 3. fetch_limited : 백프레서
=====
백프레서 비동기 소요 시간: 0.65초
```

STEP 4. 타임아웃 — 무한정 기다리지 않기

```
=====
STEP 4. fetch_with_timeout : 타임아웃
=====
타임아웃 비동기 소요 시간: 0.65초
```

STEP 5. 재시도 — 지수 백오프(exponential backoff)

```
=====
STEP 5. fetch_retry : 재시도
=====
9 실패, 0.15초 후 재시도
4 실패, 0.15초 후 재시도
1 실패, 0.15초 후 재시도
16 실패, 0.15초 후 재시도
25 실패, 0.15초 후 재시도
36 실패, 0.15초 후 재시도
49 실패, 0.15초 후 재시도
9 실패, 0.3초 후 재시도
4 실패, 0.3초 후 재시도
1 실패, 0.3초 후 재시도
재시도 비동기 소요 시간: 1.56초
```

STEP 6. 예외 격리 — 하나 실패해도 전체는 살리기

```
=====
STEP 6. 예외 격리 및 완성 수집기
=====
🔖 [시작] 총 60건의 데이터 수집을 시작합니다. (동시 요청 제한: 10개)
16 실패, 0.15초 후 재시도
25 실패, 0.15초 후 재시도
36 실패, 0.15초 후 재시도
49 실패, 0.15초 후 재시도
=====
📊 [결과 요약] 수집 완료!
✅ 성공: 60 건
❌ 최종 실패: 0 건
=====
🕒 총 소요 시간: 1.86초
```

종합 실습 1. 비동기 ETL 파이프라인

STEP 0 ~ 2. 테스트 점진적 접근(폴더 구조 > Transform > 테스트 코드)

```
platform win32 -- Python 3.11.0, pytest-9.1.1, pluggy-1.6.0 -- D:\skala\workspace\skala-python\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: D:\skala\workspace\skala-python\Python_data_0720_Day1\total1
collected 3 items

test_pipeline.py::test_카테고리_소문자화 PASSED [ 33%]
test_pipeline.py::test_음수_가격_거부 PASSED [ 66%]
test_pipeline.py::test_유효_무효_건수_일치 PASSED [100%]

===== 3 passed in 0.15s =====
```

STEP 3 ~ 6. Extract, Load, Parquet 라운드 트립 + 테스트 코드

```

async def mock_fetch(item_id: int):
    """
    - 기본 0.05초 소요
    - DEAD_TARGETS: 영구적 오류 발생 (모든 재시도 실패)
    - FAIL_TARGETS: 첫 번째 시도에서만 실패, 재시도 시 성공
    """
    await asyncio.sleep(0.05)

    # 1. 영구 실패 대상
    if item_id in DEAD_TARGETS:
        raise Exception(f"500 Internal Server Error (ID: {item_id})")

    # 2. 일시적 실패 대상
    if item_id in FAIL_TARGETS and item_id not in failed_items:
        failed_items.add(item_id)
        raise Exception(f"503 Service Unavailable (ID: {item_id} 첫 시도 실패)")

    # 3. 정상 데이터 반환
    return {
        "id": item_id,
        "name": f"Product_{item_id}",
        "category": "ELECTRONICS" if item_id % 2 == 0 else "FOOD",
        "price": 100.0 * item_id,
    }

```

```

# =====
# 3. ETL 파이프라인 함수
# =====
async def extract(ids: list[int], max_concurrent: int = 10) -> tuple[list[dict], list[dict]]:
    sem = asyncio.Semaphore(max_concurrent)

    async def one(i):
        async with sem:
            for attempt in range(3):
                try:
                    return await mock_fetch(i)
                except Exception as e:
                    if attempt == 2:
                        return {
                            "id": i,
                            "error": str(e),
                            "failed_at_attempt": attempt + 1,
                        }
            await asyncio.sleep(0.1 * (2**attempt))

    results = await asyncio.gather(*[one(i) for i in ids], return_exceptions=True)

    success = [r for r in results if isinstance(r, dict) and "error" not in r]
    dead_letters = [r for r in results if isinstance(r, dict) and "error" in r]

    return success, dead_letters

```

```

def load(valid: list[Product], out_dir=OUTPUT_DIR) -> pd.DataFrame:
    target_dir = Path(out_dir)
    target_dir.mkdir(parents=True, exist_ok=True) #

    df = pd.DataFrame([v.model_dump() for v in valid])
    df.to_csv(target_dir / "products.csv", index=False) #
    df.to_parquet(target_dir / "products.parquet", index=False) #
    return df

def save_dead_letters(dead_letters: list[dict], out_dir=OUTPUT_DIR):
    if not dead_letters:
        return

    target_dir = Path(out_dir)
    target_dir.mkdir(parents=True, exist_ok=True) #
    file_path = target_dir / "dead_letter.json" #

    with open(file_path, "w", encoding="utf-8") as f:
        json.dump(dead_letters, f, ensure_ascii=False, indent=4)

# Parquet 라운드트립 테스트 (재로딩 동일)
def test_parquet_round_trip(tmp_path): # tmp_path = pytest 임시 폴더
    df = pd.DataFrame({"id": [1, 2], "price": [10.5, 20.0]})
    p = tmp_path / "test.parquet"
    df.to_parquet(p, index=False)
    back = pd.read_parquet(p)
    pd.testing.assert_frame_equal(df, back) # ★ 완전히 같아야 통과

# extract() 비동기 수집 & 백프레셔/재시도 테스트
@pytest.mark.asyncio
async def test_extract_success_and_retry_behavior():
    # 16번은 FAIL_TARGETS (첫 시도 실패 -> 재시도 성공)
    # 1번은 DEAD_TARGETS (3회 재시도 모두 실패)
    # 2번은 정상 대상
    target_ids = [1, 2, 16]

    raw, dead_letters = await extract(target_ids)

    # 추출 성공 데이터 검증 (2, 16번)
    extracted_ids = [r["id"] for r in raw]
    assert set(extracted_ids) == {2, 16}

    # Dead Letter 데이터 검증 (1번)
    assert len(dead_letters) == 1
    assert dead_letters[0]["id"] == 1
    assert dead_letters[0]["failed_at_attempt"] == 3

```

```
# Load() 데이터 저장 및 포맷 변환 테스트
def test_load_creates_csv_and_parquet(tmp_path):
    from models import Product

    # Mock Valid Product 객체 생성
    valid_products = [
        Product(id=10, name="TestA", category="FOOD", price=50.0),
        Product(id=20, name="TestB", category="ELECTRONICS", price=150.0),
    ]

    # tmp_path(pytest 제공 임시 디렉토리)에 저장 실행
    out_dir = str(tmp_path / "test_output")
    df = load(valid_products, out_dir=out_dir)

    # 1. 반환된 DataFrame 행 개수 확인
    assert len(df) == 2

    # 2. 파일 생성 여부 확인
    csv_file = tmp_path / "test_output" / "products.csv"
    parquet_file = tmp_path / "test_output" / "products.parquet"

    assert csv_file.exists()
    assert parquet_file.exists()

    # 3. CSV 저장 내용 검증 (category 소문자 처리 확인)
    loaded_df = pd.read_csv(csv_file)
    assert loaded_df.iloc[0]["category"] == "food"
```

```
# 3. save_dead_letters() JSON 저장 테스트
def test_save_dead_letters_writes_json(tmp_path):
    mock_dead_letters = [
        {"id": 1, "error": "500 Error", "failed_at_attempt": 3},
        {"id": 4, "error": "500 Error", "failed_at_attempt": 3},
    ]

    out_dir = str(tmp_path / "test_output")
    save_dead_letters(mock_dead_letters, out_dir=out_dir)

    json_file = tmp_path / "test_output" / "dead_letter.json"

    # 1. JSON 파일 생성 여부 확인
    assert json_file.exists()

    # 2. 저장된 JSON 데이터 내용 검증
    with open(json_file, "r", encoding="utf-8") as f:
        saved_data = json.load(f)

    assert len(saved_data) == 2
    assert saved_data[0]["id"] == 1
    assert saved_data[1]["id"] == 4
```

```
JNU+175072@DESKTOP-J3F7C64 MINGW64 /d/skala/workspace/skala-python/Python_data_0720_Day1/total1 (main)
$ pytest -v
===== test session starts =====
platform win32 -- Python 3.11.0, pytest-9.1.1, pluggy-1.6.0 -- D:\scala\workspace\scala-python\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: D:\scala\workspace\scala-python\Python_data_0720_Day1\total1
plugins: asyncio-1.4.0
asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 7 items

test_pipeline.py::test_category_lowercase PASSED [ 14%]
test_pipeline.py::test_negative_price_rejected PASSED [ 28%]
test_pipeline.py::test_valid_invalid_count PASSED [ 42%]
test_pipeline.py::test_parquet_round_trip PASSED [ 57%]
test_pipeline.py::test_extract_success_and_retry_behavior PASSED [ 71%]
test_pipeline.py::test_load_creates_csv_and_parquet PASSED [ 85%]
test_pipeline.py::test_save_dead_letters_writes_json PASSED [100%]

JNU+175072@DESKTOP-J3F7C64 MINGW64 /d/skala/workspace/skala-python/Python_data_0720_Day1 (main)
$ python total1/pipeline.py
=====
비동기 ETL 파이프라인 실행 결과
=====
요청 총 건수 : 60건
추출 데이터 : 57건
유효 데이터 : 56건
오류 데이터 : 1건
저장 데이터 : 56건
실패 데이터 : 3건
```

- `extract()` 위한 `mock_fetch` 추가
- `load()` : 데이터 csv, parquet 저장
- `save_dead_letters` : dead_letters 격리 및 기록

STEP 7. Ruff 로 마무리

```
JNU+175072@DESKTOP-J3F7C64 MINGW64 /d/skala/workspace/skala-python/Python_data_0720_Day1 (main)
$ python -m ruff check .
F401 [*] `typing.Annotated` imported but unused
--> practice2\solution2.py:4:20
|
| 2 | import re
| 3 | from pydantic import BaseModel, Field, ValidationError, field_validator
| 4 | from typing import Annotated, List
|   |         ~~~~~~
| 5 |
| 6 | file_path = 'data/api_response.json'
|
help: Remove unused import: `typing.Annotated`

Found 5 errors.
[*] 2 fixable with the `--fix` option (3 hidden fixes can be enabled with the `--unsafe-fixes` option).
(.venv)
JNU+175072@DESKTOP-J3F7C64 MINGW64 /d/skala/workspace/skala-python/Python_data_0720_Day1 (main)
$ python -m ruff check --fix .
F841 Local variable `results` is assigned to but never used
--> practice3\solution3.py:106:5
|
| 104 |     sem = asyncio.Semaphore(MAX_CONCURRENT) # 입장권 10장
| 105 |
| 106 |     results = await asyncio.gather(*[fetch_retry(i, sem) for i in range(60)])
|   |     ~~~~~~
|
help: Remove assignment to unused variable `results`

Found 5 errors (2 fixed, 3 remaining).
No fixes available (3 hidden fixes can be enabled with the `--unsafe-fixes` option).
(.venv)
JNU+175072@DESKTOP-J3F7C64 MINGW64 /d/skala/workspace/skala-python/Python_data_0720_Day1 (main)
$ ruff format .
7 files reformatted, 1 file left unchanged

JNU+175072@DESKTOP-J3F7C64 MINGW64 /d/skala/workspace/skala-python/Python_data_0720_Day1 (main)
$ ruff check .
All checks passed!

JNU+175072@DESKTOP-J3F7C64 MINGW64 /d/skala/workspace/skala-python/Python_data_0720_Day1 (main)
$ python -m ruff check .
E712 Avoid equality comparisons to `True`; use `r.get("ok"):` for truth checks
--> advaced\dead_letter.py:136:57
|
| 135 |     # 결과 통계 분류
| 136 |     ok = [r for r in results if isinstance(r, dict) and r.get("ok") == True]
|   |                                     ~~~~~~
| 137 |     fail = [r for r in results if isinstance(r, dict) and r.get("ok") == False]
|
help: Replace with `r.get("ok")`

E712 Avoid equality comparisons to `False`; use `not r.get("ok"):` for false checks
--> advaced\dead_letter.py:137:59
|
| 135 |     # 결과 통계 분류
| 136 |     ok = [r for r in results if isinstance(r, dict) and r.get("ok") == True]
| 137 |     fail = [r for r in results if isinstance(r, dict) and r.get("ok") == False]
|   |                                     ~~~~~~
| 138 |
| 139 |     # 🟡 최종 실패 건을 버리지 않고 dead_letter.json 파일로 기록
|
help: Replace with `not r.get("ok")`
```

- 'True', 'False'로 바뀌어야 ruff passed이나 잘못된 결과 나오므로 무시

추가 과제

1. tracemalloc

```
=====
[메모리 사용량 비교 측정 시작]
=====

[방식 1: readlines() - 전체 로드]
- 처리된 레코드 수: 200,000 개
- Peak 메모리 사용량: 27.53 MB (28193.00 KB)

[방식 2: Generator - 한 줄씩 스트리밍]
- 처리된 레코드 수: 200,000 개
- Peak 메모리 사용량: 0.05 MB (51.31 KB)
```

- readlines와 제너레이터 메모리 비교

2. dead_letters

```
JNUH175072@DESKTOP-J3F7C64 MINGW64 /d/skala/workspace/skala-python/Python_data_0720_Day1 (main)
$ python advanced/dead_letter.py

=====
한 걸음 더. dead_letter 격리
=====
[시작] 총 60건의 데이터 수집을 시작합니다. (동시 요청 제한: 10개)
9 실패, 0.15초 후 재시도
4 실패, 0.15초 후 재시도
1 실패, 0.15초 후 재시도
16 실패, 0.15초 후 재시도
25 실패, 0.15초 후 재시도
36 실패, 0.15초 후 재시도
49 실패, 0.15초 후 재시도
9 실패, 0.3초 후 재시도
4 실패, 0.3초 후 재시도
1 실패, 0.3초 후 재시도

=====
[결과 요약] 수집 완료!
[성공: 57 건]
[최종 실패: 3 건]
=====
총 소요 시간: 1.57초
```

- 실패 데이터 격리 및 json 기록